Aurevia System Design: Voice Sales Agent Platform

Oct 15, 2026 · @Virat

Purpose, goals and principles

This document describes how Aurevia is built and how it must evolve to meet the master plan: a multi-tenant AI voice agent that cold calls, qualifies, books and follows up, sounds like a friendly human call, and costs far less than the sales team it replaces. It records what exists today (from the repository) and what is new (from the master plan), and marks each as **exists** or **new**.

The companion document is the Aurevia Master Plan, which says what to build and in what order. This one says how the parts fit together.

Design goals (targets marked *proposed* are for Virat to confirm)

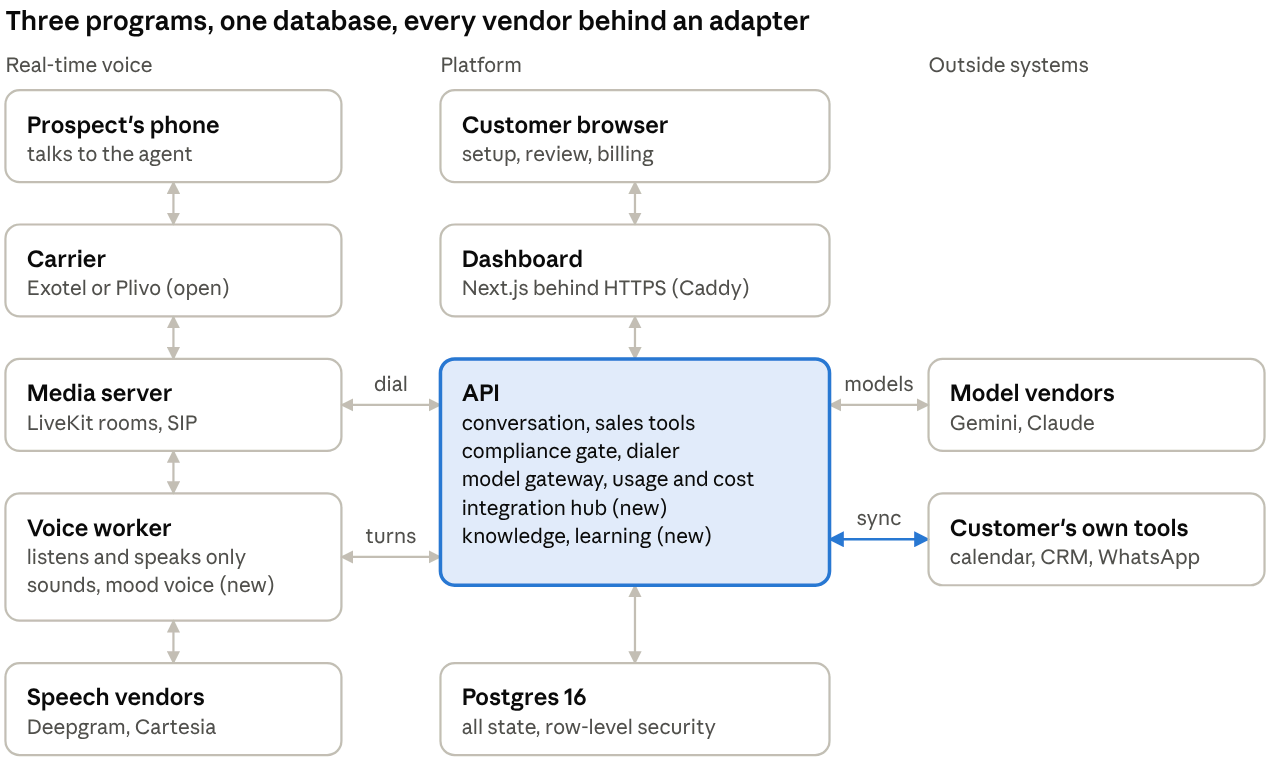
Goal	Target
Reply speed, end of prospect speech to first agent audio	median under 1.0 s, p95 under 1.8 s (from the project's own docs)
Variable cost per talk minute	2 rupees or less (estimate today: 3 to 5)
Tenant isolation	no cross-tenant read or write, enforced in the application and in the database
Honesty	the agent never claims an action a tool did not complete, and says it is an AI whenever asked
Outbound compliance	no outbound call without an allowed gate decision recorded first
Pilot capacity	20 simultaneous calls on one server (*proposed*)
Pilot availability	99% of call attempts set up successfully (*proposed*)

Design principles

1. **The model is not the controller.** The model proposes; deterministic code decides. Side effects happen only through validated, tenant-scoped, audited tools, and a state machine owned by the server moves the call through its stages.
2. **Speak first, record later.** The prospect hears a reply as soon as it is ready. Bookkeeping (stage, mood, facts) runs after the first word, never before it.
3. **Every vendor sits behind an adapter.** Model, speech-to-text, text-to-speech, telephony, calendar, CRM and messaging can each be swapped by configuration. A second vendor per voice layer exists for failover.
4. **Fail closed.** Unknown means blocked: an unknown do-not-disturb status, an unreviewed policy, a missing consent, a missing tenant context. Failing open is never the default.
5. **Isolation twice.** Services filter by the caller's tenant, and Postgres row-level security backs that up.
6. **Measure before tuning.** Every latency, cost and quality change is justified by a saved benchmark, and reports separate mocked tests, real-provider tests, synthetic audio and human calls.
7. **Cost is a first-class metric.** Every provider call records its usage, and every call has a computed cost.
8. **Humans approve what the agent learns.** No technique reaches a live call without an audited human approval and a versioned playbook.
9. **Natural, never deceptive.** Warmth, timing, listening sounds and emotion make the call pleasant. They are never used to claim the agent is a person.

Architecture at a glance

Aurevia is three programs (the voice worker, the API and the dashboard) sharing one Postgres database. Everything outside them is a vendor behind an adapter, so any vendor can be replaced by changing configuration.



The blue box is the API; (new) and the blue arrow mark work added by the master plan. Everything else exists today.

architecture overview · 3 programs, 1 database

A phone call travels down the left column: the prospect, the carrier, the media server, then the voice worker, which uses the speech vendors. The worker never decides what to say. For every turn it asks the API, which builds the prompt, runs the tools and asks the model vendors through the gateway. The dashboard and the customer's browser reach the API through the HTTPS front door. The API alone writes to the database, and every request carries a tenant, so one customer can never see another's data.

Components and responsibilities

The system is three deployable programs (API, voice worker, dashboard) plus the services they depend on. Each row below owns one job; the rule is that no component does another's job.

Component	Job	Technology	Status
Dashboard	Tenant interface: setup, leads, campaigns, call review, analytics, billing	Next.js 15, TypeScript, Tailwind	exists; knowledge base, analytics, billing are new
API	Sign-in, tenants, leads, campaigns, calls, compliance, usage, platform admin	FastAPI, async SQLAlchemy, Alembic	exists
Conversation engine	Builds each turn's prompt, runs the tool loop, streams the reply	Python module in the API (`conversation/`)	exists
Sales control	Stage state machine, validated tools, scheduling	`sales/` , `tools/`	exists
Model gateway	One interface to every model vendor: deadlines, fallback, circuit breaker, usage record	`gateway/` , `providers/`	exists; routing is new
Voice worker	Listens, detects turns, handles interruptions, speaks; never decides what to say	LiveKit Agents, Silero VAD	exists
Speech adapters	Speech-to-text and text-to-speech vendors	Deepgram, Cartesia	exists; second vendor per layer is new
Backchannel engine	Plays short listening sounds while the prospect speaks	Voice worker module	new

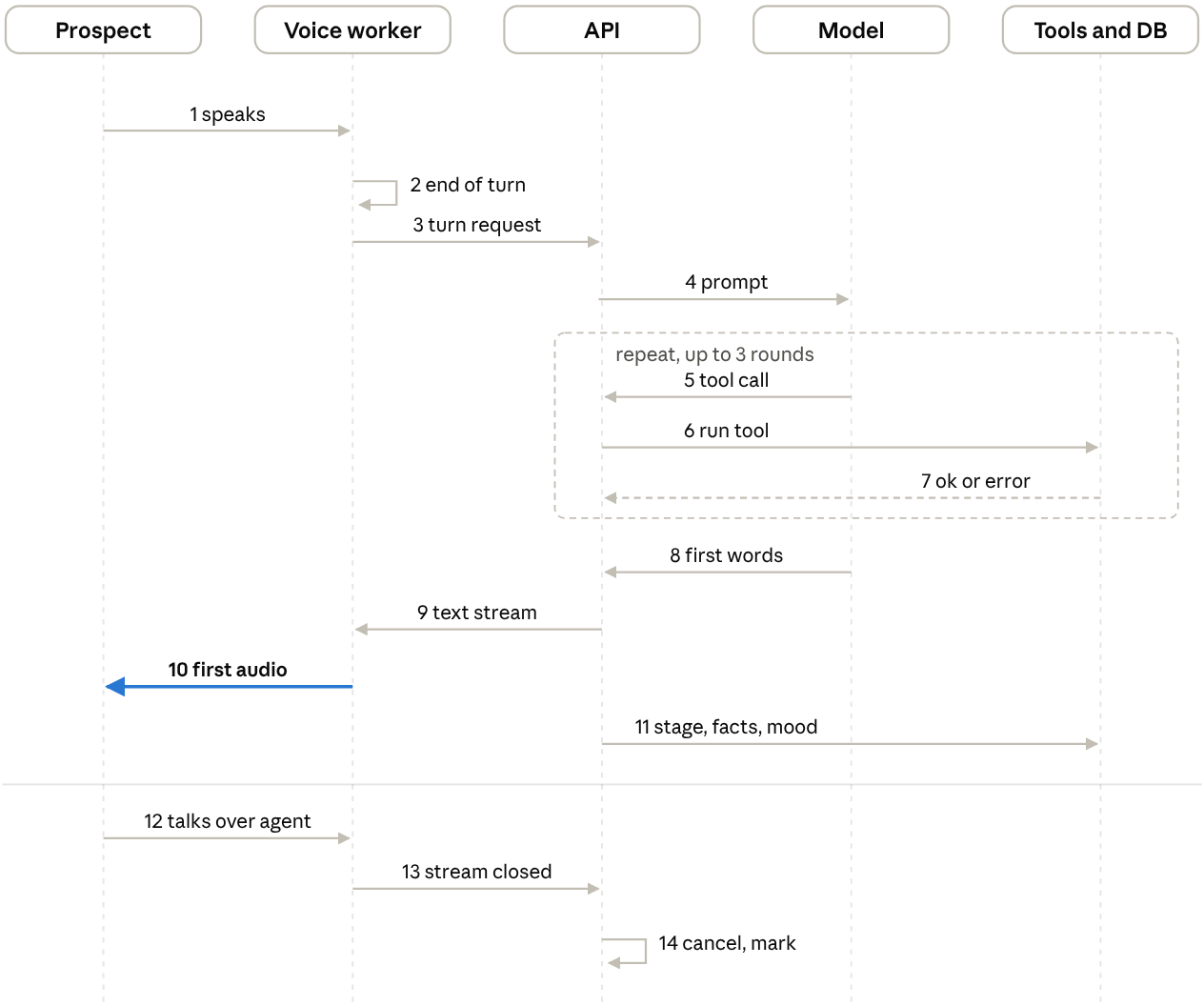
Component	Job	Technology	Status
Mood and voice controller	Turns the prospect's mood into speed and emotion settings	Worker and API	new
Audio cache	Pre-rendered clips (greeting, listening sounds, closings)	Files or object storage	new
Media server and phone bridge	Rooms for audio, SIP link to the phone network	LiveKit server and SIP, Redis	exists; phone path untested
Carrier	The phone network connection	Exotel or Plivo (decision open)	open
Compliance gate	Allows or blocks every outbound call and records why	`compliance/`	exists; real do-not-disturb lookup is new
Campaign dialer	Lead queue and pacing	`campaigns/`	exists; off by default
Memory service	Facts remembered across calls	`memory/`	exists
Knowledge service	Per-tenant documents and FAQs, retrieved per turn	API module	new
Learning service	Recordings to transcripts, labels, timing profiles, playbooks	API module and worker jobs	new
Integration hub	Calendar, CRM and messaging adapters with a reliable outbox	API module and worker jobs	new
Usage and cost engine	Metering, price table, plan limits, budget guards	`usage/`	exists; phone cost and cache savings are new
Postgres 16	All durable state, with row-level security	PostgreSQL	exists

Component	Job	Technology	Status
Redis	Coordination for the media server and phone bridge	Redis	exists
Object storage	Temporary audio and exports	S3-compatible, Indian region	new, only for recordings
HTTPS front door	TLS for the dashboard, API and media address	Caddy	planned
Observability	Logs, traces, metrics, alerts	to be chosen	new

One turn, end to end

A turn is one exchange: the prospect speaks, the agent answers. The sequence below shows who does what, and where the time goes.

The prospect hears the first words before the bookkeeping starts



Target: step 10 within 1.0 s of the prospect finishing step 1. Steps 12 to 14 are an interruption.

one turn · 5 parties, 14 steps

Steps 1 to 3. The worker listens through voice activity detection and streaming speech-to-text. When it judges the prospect has finished (step 2), it sends the text to the API as a turn request. With preemptive generation this request can start on a stable partial transcript and be cancelled if the prospect keeps talking.

Steps 4 to 7 (the tool loop). The API builds the prompt and asks the model through the gateway. If the model asks for a tool, such as free meeting slots, the API runs it with validated arguments in a savepoint, writes an audit row and returns `ok` or an error. This repeats at most three times, and the last round offers no tools.

Steps 8 to 10. The first words stream back to the worker, which sends them to text-to-speech so the prospect hears them. This is the number to keep under one second.

Step 11. Only now does the bookkeeping pass record the stage, qualification facts, objections and mood. A failure here never affects the call. The mood can produce a `voice` control event that changes speed or emotion for the next utterance.

Steps 12 to 14 (interruption). When the prospect talks over the agent, the worker stops the audio and closes the stream. The API sees the closed stream, cancels the generation and records the turn as interrupted. Nothing from the cancelled reply is saved as said.

Latency budget

The measured median from the end of the prospect's speech to the first agent audio is about 2.8 s (p95 about 6.5 s). The target is 1.0 s median. The stages overlap, so the "today" figures below were measured separately and must not be added up.

Stage	Today (median)	Budget (median)	Main technique
Waiting to be sure the prospect has finished	579 ms	250 ms	Adaptive endpointing or a turn-detection model
Speech-to-text final transcript	486 ms	100 ms	Start work on the stable interim transcript, not the final one
Backend: prompt build and state read	not measured	30 ms	Cache the tenant prompt parts; read state once
Model first token	1,668 ms	400 ms	Smaller default model, prompt caching, fewer tool rounds
Text-to-speech first byte	170 ms	120 ms	Warm streaming connection, start on the first clause
Network, buffering, phone jitter	not measured	100 ms	India-region hosting, warm connections

Stage	Today (median)	Budget (median)	Main technique
Total	**2,827 ms (measured end to end)**	**about 1,000 ms**	

How the gap closes

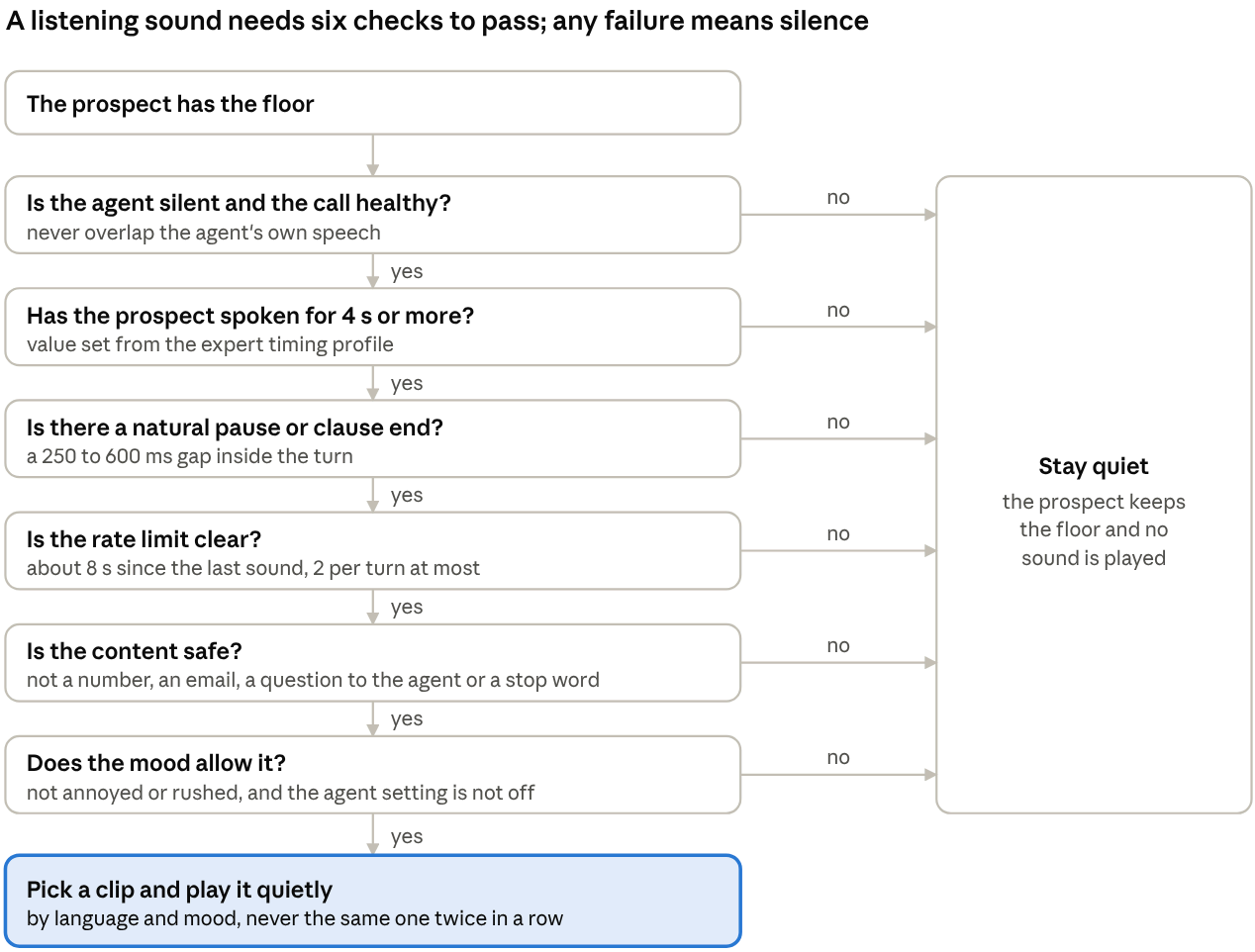
1. **Preemptive generation.** Start the model when the interim transcript looks complete. If the prospect keeps talking, cancel the generation and discard it. The installed LiveKit version already supports this; it is on for the model and off for speech.
2. **Preemptive speech and clause streaming.** Send the first four to six words to text-to-speech as soon as they arrive, rather than a full sentence.
3. **A smaller prompt.** About 4,000 input tokens per turn is the first thing to cut. Split the prompt into a stable part (honesty rules, company, setup), which providers can cache, and a small changing part (stage, notes, last turns).
4. **Fewer model rounds.** Each tool round costs a full model trip. Pre-fetch free meeting slots when the call reaches the next-step stage so a booking needs one round, and allow independent tools to run in the same round.
5. **Model choice by turn.** A fast small model answers routine turns; a stronger one answers objections, pricing and booking (see the model routing section).
6. **Bridge sounds.** If the model has not produced a first token within a threshold (proposed 700 ms), play one pre-rendered neutral sound such as "mm". It makes no claim and says nothing about content. At most one per turn, and never before a question.
7. **Warm paths.** Keep long-lived connections to the model and speech vendors, and keep the worker's models loaded, so no turn pays a cold start.
8. **Co-location.** Run the worker, API and media server on the same host or network. Measure from India and from a region nearer the vendors before choosing, because the hosting plan expects the vendors to be outside India.

Every stage is already or must be recorded per turn (`voice/metrics.py` and the worker's turn metrics), so each change is judged by its own number.

Human-feel subsystem

Three mechanisms make the call feel like two friends talking: short listening sounds while the prospect speaks, a voice that follows the prospect's mood, and turn-taking that knows when to wait. All three are **new**. They live in the voice worker or in control events, never in the main reply path, so a failure in any of them cannot delay or break a reply.

Listening sounds (the backchannel engine). A small module in the worker (`backchannel.py`) watches the prospect's voice activity and partial transcript. It runs about every 100 ms while the prospect has the floor, and plays a sound only when every check in the drawing passes.



listening-sound decision · 6 checks

How a clip plays without taking the turn is the open question that WP-11.1 settles. Whatever method wins must keep the prospect's turn intact, must not let the worker treat its own clip as the prospect's speech, and must be tested for echo on a real phone line.

Parameter	Default	Tuned from
Prospect speaking time before the first sound	4 s	Expert timing profile
Natural pause window inside a turn	250 to 600 ms, always shorter than the end-of-turn wait	Expert timing profile
Gap between sounds	about 8 s, randomised by 25%	Expert timing profile
Most sounds in one prospect turn	2	Expert timing profile
Volume	slightly below the agent's normal voice (*proposed*: 85%)	Listening tests
Level	off, low (default) or medium, set per agent	Customer choice
Clips	4 to 6 per voice and language, pre-rendered once and cached	Chosen voice

Example clips: English "hmm", "okay", "right", "yeah"; Hindi "haan", "achha", "theek hai", "ji". Because they are cached, each use costs nothing. If the engine raises any error it switches itself off for the rest of that call, and logs only the error category.

Mood and voice. The bookkeeping pass labels the prospect's mood with a confidence after each turn, from their words first. The API sends a `voice` control event and the worker updates the text-to-speech settings for the next utterance.

Mood	Voice setting	Wording hint
Warm	Normal speed, lighter emotion	Match their energy, keep it short
Neutral	Default	Default
Rushed	About 10% faster	Shortest replies, offer a callback early
Annoyed	Slightly slower and softer	Apologise once, offer to end or call back
Skeptical	Steady pace, plain tone	Facts, no hype
Confused	About 10% slower	Simpler words, one idea at a time

Rules: change a setting only after the same mood at confidence 0.6 or higher on two turns in a row; return to default when the topic changes; keep speed within 15% of default and confirm the ranges the vendor accepts in the installed plugin; never use pressure tactics when the prospect is annoyed. Voice from tone of audio, rather than words, is a later option.

Turn-taking.

- *End-of-turn wait.* Adaptive within a minimum and a maximum: shorter after a yes-or-no question, longer after an open question or when the prospect says "let me think". Benchmarked against fixed timing first (WP-10.4).
- *The prospect's own listening sounds* ("hmm", "yeah") must not interrupt the agent. Require at least two words, or use LiveKit's adaptive interruption setting, so the agent keeps talking.
- *Real interruptions.* Stop the audio at once, keep the prospect's words, answer them, and do not finish the old sentence (exists).
- *False interruptions* such as a cough: resume where the agent stopped. The installed LiveKit version supports this; it needs enabling and a benchmark.
- *"Hold on".* Say "sure", stay silent for a set time, then check in once. Reuses the existing silence tracker, which also handles long silences with re-prompts and a polite goodbye.

Measuring it. Per call: listening sounds per minute, share of turns with one, overlap incidents, false interruptions, interrupted-turn rate, end-of-turn wait, words per reply. These are compared with the expert numbers from the timing profile, then confirmed by the blind listening test in Gate 11.

Conversation engine, sales control and tools

The voice worker asks the API for each reply (`/internal/v1/calls/{id}/turns` , streamed as newline-delimited JSON). Everything about what the agent says and does lives behind that endpoint.

Prompt assembly (exists, `conversation/prompt.py`). The system prompt is built only from the tenant's validated agent settings, the server-owned sales state and server-held lead data. Nothing the prospect says enters the system prompt. Remembered facts are flattened to one line each and framed as notes, not instructions. The honesty rules are fixed text and cannot be edited by a tenant.

Prompt layers (new order and budget). Layers are ordered so the first ones are identical on every turn and can be cached by the model provider:

1. Fixed honesty and speaking rules
2. Tenant company profile and setup (personality, qualification questions, objection and escalation guidance)
3. Retrieved company knowledge (at most a few short chunks, new)
4. Playbook techniques (at most three, new)
5. Prospect details and memory notes
6. Current stage goal and mood hint
7. Recent turns plus a running summary of earlier ones

Proposed budget: stable layers 1 to 2 up to 1,200 tokens, layers 3 to 6 up to 800, layer 7 up to 800. That is at most 2,800 input tokens per turn, against about 4,000 today.

Sales state machine (exists, `sales/state.py`). Eleven states: new, opening, discovery, qualification, pitch, objection, next step, follow-up, meeting booked, human handoff, completed. Only the allow-listed transitions can happen. The server moves the first two; after that the model may propose a change through the `set_stage` tool, which the server validates. Each state has a one-line goal that goes into the prompt.

Tools (exists, `tools/`). Twelve: `set_stage`, `qualify_lead`, `mark_interested`, `mark_not_interested`, `log_objection`, `add_note`, `schedule_followup`, `get_available_slots`, `book_meeting`, `cancel_meeting`, `flag_for_handoff`, `request_do_not_call`. The executor validates arguments, checks that a lead exists when the tool needs one, runs the tool in a database savepoint, writes an audit row, and returns `{ok, error}`. The same request twice in a call is answered from the audit row, so nothing happens twice. New tools: `send_followup` (template-based, consent-checked), `lookup_knowledge`, and calendar and CRM actions behind the same executor.

Tool loop rules.

- At most three tool rounds per turn; the last round offers no tools.
- If the first output of a round is a tool call, nothing is spoken until the result is back. Text before a tool call may only be a short bridge that claims nothing.
- The prompt and the engine both state that nothing is booked, saved or sent unless the tool result says `ok` is true.

Speak first, record later (exists). After the first words of a reply stream out, a separate bookkeeping pass records stage, qualification facts, objections, interest and, new, the prospect's mood. A failure there never affects the call.

Model routing (new). A deterministic router picks the model per turn from signals the server already has: the stage (objection and next step use the stronger model), whether the last turn used a tool, and flags in the prospect's words (price, "not interested", a long question). Everything else uses the fast model. Both receive the same prompt, so the persona stays steady. Routing rules are tested with the sales scenario suite before they are turned on.

Memory (exists, `memory/`). After a call with a known lead, the model proposes facts as JSON through the gateway. Each is validated for kind, length and confidence; facts at or above the confidence threshold (0.6) go into later prompts. A person can delete a fact.

Language. The agent replies in the prospect's language and defaults to the agent's setting. Per-turn language detection switches the text-to-speech language (exists for the switch itself; detection quality for Hinglish is to be benchmarked).

Call flow: goals, objections and calls that never reach a person

This section is **new**. It adds three things to the conversation engine: a goal ladder, a structured way to handle objections, and early detection of calls that never reach a decision maker.

Goal ladder. The sales state gets one extra field, `goal_level`, with four values: `meeting`, `stay_in_touch`, `referral`, `end`. It starts at `meeting`. Each time the prospect declines, the prompt layer moves it down one step. The agent may accept "send me an email" only after the `meeting` step has been tried once. A `request_dnc` call (the existing do-not-call tool) overrides everything: the call ends politely, the number is blocked through the compliance gate, and no later step can reopen it. No new sales stages are needed.

Objection handling. `log_objection` gains a `kind` field (`price`, `timing`, `authority`, `need`, `other`) and an `isolated` flag set once the prospect confirms it is the only blocker. The approved playbook entries tell the model the order: agree, ask why, isolate, then realign, educate or disqualify. The reply is always a question, and the model is told never to argue against the prospect's premise.

Tone by stage. The stage and the last objection pick the tone, using the mood table above, and the voice worker applies it as a `voice` control event. Starting values: warm at the opening, calm and slower in discovery, empathetic when a problem is shared, steady and confident at price. These are tuned by ear in Phase 11, not fixed.

Early call classification. In the first seconds the worker labels what it hears and acts on it:

What is heard	How it is detected	Action
Voicemail greeting	Carrier answering-machine signal if available, plus a transcript check for phrases like "leave a message"	Hang up within about 3 seconds by default, or leave a 15-second message if the campaign allows it; log `voicemail`
Phone menu	Phrases like "press" or "extension"	Send the extension if one is stored; otherwise end the call and log `menu`; never wait through a long menu
Receptionist	A person answering with a company name or "how may I help"	Ask for the contact by name, give a short reason only if asked, do not pitch; if told the person is out, ask for the best time and log it
Wrong number	"Wrong number" or "who is this" with no match	Apologise, end the call, mark the number invalid and stop the sequence
The right person	Anything else	Run the opening

The label is stored on the call record, so reports can show how many dials reached a person. This matters for cost: in one sample of 50 dials only about 3 became real conversations, so a call that wastes 30 seconds on a menu costs real money across thousands of dials.

Calling windows and retries. The dialer, which is off by default, picks the next attempt time from the prospect's local time zone. The first window to test is 3 to 5 pm, which is one practitioner's rule and needs measuring on our own data. The compliance gate's legal calling hours always win over any window. Retries (proposed): a no-answer is tried again on a different day and time; "call me in a week" is honoured as a scheduled follow-up; a fixed maximum number of attempts per lead; wrong numbers and refusals are never retried.

Model gateway, speech adapters and failover

Model gateway (exists, `gateway/`, `providers/`). One interface for every model vendor. It sets a first-token deadline and a total deadline, and records for every reply the model requested, the model that served it, token counts, time to first token and whether the prospect interrupted. Vendor names appear only in `providers/registry.py`; a test enforces that no other code imports a vendor SDK. Choosing Gemini or Anthropic is configuration. Model names are pinned, never "latest".

- **Normalised errors.** Every adapter raises one of `auth_failed`, `rate_limited`, `invalid_request`, `unavailable`, `timeout`. Vendor error text, which can echo request data, is never kept.
- **Fallback.** A list of backup models is tried when one fails before its first word. A circuit breaker skips a failing model for 60 seconds.
- **Secrets.** Keys are secret-typed settings, never returned by an endpoint, and redacted from logs and tracebacks.

Gateway additions (new).

- *Routes.* Two named routes, `fast` and `strong`, each with a primary and fallbacks. The conversation engine asks for a route, not a vendor.
- *Deadlines per route* (proposed): the fast route's first-token deadline 1.5 s before falling back, the strong route's 2.5 s; total 8 s for both.
- *Cache awareness.* The request carries a stable-prefix marker so adapters can use each vendor's prompt caching, and the usage record stores cached and uncached tokens separately.
- *Streaming contract.* The turn stream carries text deltas, tool events and a final `done` with the new sales state. New control events, `mood` and `voice`, let the API tell the worker to change speed or emotion for the next utterance. Older workers ignore unknown event types.

Speech adapters (exists, `voice-worker/.../speech_providers.py`). This is the only worker module that imports speech vendor plugins; the runtime uses LiveKit's `stt.STT` and `tts.TTS` interfaces. Vendor keys exist only in the worker container, never in the API, and the worker holds no model key.

Speech failover (new).

- Add a second speech-to-text and a second text-to-speech vendor behind the same adapters. LiveKit Agents has fallback adapters for both; confirm them in the installed version before relying on them.
- Choose the vendor at call start from the circuit breaker's health state, and fail over inside a call for text-to-speech (one retry on a pre-chosen fallback voice for that language).
- Failures are classified from error type and status only (`credits_exhausted`, `rate_limited`, `timeout`, `provider_unavailable`, `unknown`); provider text is never read or logged.
- **Phone calls are never left in silence.** After repeated failures the worker plays a pre-rendered apology line ("I'm having trouble hearing you, a colleague will call you back"), schedules a callback for the lead and ends the call. Browser calls keep going so audio can be retried, as they do today.

What a tenant can and cannot choose. A tenant picks voice, language, personality and company content. A tenant never picks the vendor, never supplies a vendor key, and never edits the honesty rules. Only the platform operator changes vendors, routes and prices.

Data model and tenancy

All durable state is in one Postgres 16 database, migrated by Alembic (eight migrations so far; the exact tables are in `docs/03-platform/database-schema.md`). The table below groups records by domain and lists what the master plan adds.

Domain	Existing records	New records
Identity and team	tenants, users, memberships, custom roles, invitations, refresh sessions, audit events with a hash chain	API keys with scopes, two-factor secrets
Agents and calls	agent settings, calls, call turns, turn metrics, usage events, transcripts	mood and voice setting per turn, techniques offered per turn, cost summary per call

Domain	Existing records	New records
Sales	leads, qualification state, appointments, scheduling settings, tool executions (audit and idempotency)	external calendar event id, CRM record id
Memory	extracted facts with kind and confidence	none
Campaigns	campaigns, campaign leads, queue state	pacing and daily caps per campaign
Compliance	consents, do-not-call list, gate decisions, policy versions, tenant policy overrides	do-not-disturb lookup cache
Telephony	numbers, test numbers	carrier trunk settings per environment
Usage and plans	usage events, plans, prices	price versions (immutable), cost events, budget alerts, invoices
Knowledge	none	knowledge sources, chunks (with search index)
Learning	none	learning sources, redacted transcripts, labels, technique candidates, reviews, playbooks, playbook techniques, timing profiles
Integrations	none	connections (encrypted tokens), sync log, outbox messages, message templates, deliveries, webhook endpoints and deliveries

Rules for every tenant-owned table

1. A `tenant_id` column that cannot be empty, and an index that starts with it.
2. Forced row-level security comparing the row's tenant with the transaction's `app.tenant_id` setting (set by `aurevia.db.session.set_tenant_context`). A transaction without tenant context sees nothing.

3. The API connects as a role that is neither superuser nor `BYPASSRLS`; staging and production refuse to start otherwise.
4. Application grants have no DELETE. Retention purges run through security-definer functions.
5. Another tenant's row is reported as 404, never 403, so existence is not leaked.
6. Migrations only move forward. An applied migration is never edited.

Conventions (proposed for new tables)

- Identifiers are UUIDs; timestamps are UTC.
- Money is stored as integers (paise or micro-rupees), never floats.
- Platform-owned tables (policy versions, price versions, plans) are read-only to tenants and immutable once published.
- Secrets (carrier keys, calendar and CRM tokens) are stored encrypted with a key held outside the database, and are never returned by any endpoint.

Retention

Data	Kept for
Call transcripts	90 days, configurable, purged hourly
Redacted learning transcripts	180 days (design default)
Imported audio	deleted as soon as the transcript succeeds, at most 24 hours (*proposed*)
Audit events	kept, chained and verifiable
Consent and gate decisions	as long as the lead exists, then the period counsel sets
Live call audio	not stored today

Export and erasure of a person's data (exists for leads) extends to every new table that holds data derived from that person's calls, messages or CRM links.

Learning pipeline

This is the system behind Phase 12. It follows `docs/07-learning/sales-learning-design.md`, adds the audio step that design leaves out, and splits the result into two outputs: a timing profile that shapes how the agent sounds, and a human-approved playbook that shapes what it says. Nothing in it trains or fine-tunes a model, and nothing reaches a live call without a person's approval.

One set of recordings feeds two outputs: how the agent sounds and what it says



learning pipeline · 1 source, 2 outputs

The blue box is the one human gate. Everything before it is automated and everything after it is versioned and reversible.

Stage contracts

Stage	Takes	Produces	Rule
1 Intake	Audio and a consent record	A source record: legal basis, who gave permission, retention date	No source record, no processing. Audio is deleted once its transcript exists (at most 24 hours, *proposed*).
2 Transcribe	Audio	Transcript with speaker labels and start and end time for every word	A vendor adapter; word error rate checked by hand on three calls
3 Redact	Transcript	Transcript with placeholders such as [NAME]; a report of counts only	Patterns plus a model detector; the unredacted copy is deleted on success
4 Label	Redacted transcript	Stage spans, objection category, outcome, qualification facts, talk ratio, question count, each with a source (model or human) and a confidence	A person checks about one in five
5a Timing profile	Word times and labels	Per-stage aggregates (10th, 50th and 90th percentile)	Aggregates only; no voice fingerprints stored
5b Extract	Labelled transcripts	Candidate techniques: a short pattern, its conditions (stage, objection, language), evidence excerpts, outcome counts, confidence	Transcripts are framed as data in the extraction prompt; candidate text is length limited

Stage	Takes	Produces	Rule
6 Review	Candidates	Approved, edited or rejected, with a reason	An automatic policy check runs first (honesty, invented facts, pressure tactics, compliance); every decision is audited
7 Playbook	Approved techniques	An immutable version: draft, active, retired	One active version per agent; activation needs a passing evaluation
8 Evaluate	Candidate against the no-playbook baseline	A report per version	Only an improvement with no honesty or compliance regression can activate; then a live A/B on half of pilot calls
9 Retrieve	Stage, objection category, language	At most three techniques in the prompt	Under 50 ms; below the honesty rules, never replacing them

How the timing profile is computed. A *reply gap* is the time from the end of one speaker's last word to the start of the other speaker's first word, measured on the diarised word times. A *listening sound* is a short utterance (three words or fewer, from a lexicon such as hmm, okay, yeah, haan, achha) made by the listener. For each stage and role the profile stores: reply-gap percentiles, listening sounds per minute and the spacing between them, speech rate in words per minute, turn length, and overlap rate. Phase 11 reads it for its defaults: the speaking time before a sound, the gap between sounds, and the bounds on the end-of-turn wait.

How retrieval works. Each technique carries tags for stage, objection category and language. At the start of a call the active playbook for the agent is loaded into memory, so no database read is needed during a turn. Each turn matches on the current tags, ranks by the strength of evidence (the number of calls where it appeared and how often those calls booked; a minimum of five supporting calls is *proposed*), and adds at most three under the heading "Techniques that work for this team". The report states that this is correlation. Only the simulator and live A/B tests count as evidence that a technique helps.

Tenant scope. Every learning table has a tenant, forced row-level security and no DELETE grant except retention purges. Techniques never cross tenants. A shared global playbook would need its own legal review and anonymised, aggregated evidence, and is out of scope.

Source types. The pipeline accepts two kinds of input and treats them differently. A `call` is a real phone recording with a transcript labelled by speaker; it produces both technique candidates and timing numbers. A `talk` is an interview or lesson about selling, supplied as a transcript (or a voice recording made from it); it produces technique candidates only, because a studio conversation does not show how people pause on the phone. The timing stage refuses a `talk`. Both types go through the same review: nothing becomes usable until Virat approves it.

Seed playbook. The first entries are written by hand from the expert material, in the fixed format (situation, what is said, tone, goal, source, status) and start as `proposed`. They cover the call structure, the goal ladder, the objection method, booking, qualification and tone by stage described in the Master Plan, Phase 12.

Integrations: calendar, CRM, messaging and the public API

All of this is **new**. The existing scheduling code (`sales/scheduling.py`) already says external calendars can sit behind the same tool interface; this design uses that.

Adapter interfaces. Three protocols live in `providers/`, each returning typed results and the same normalised errors (`auth_expired`, `rate_limited`, `not_found`, `invalid`, `unavailable`):

- `CalendarProvider`: free and busy times for a range, create event, cancel event.
- `CrmProvider`: upsert contact, add activity, update deal or lead status, fetch leads.
- `MessageChannel`: send a template with parameters, receive delivery status and inbound replies.

Connections. A tenant owner connects an account through OAuth with a signed state value and PKCE. Scopes are the smallest that work (calendar: events and free/busy only). Tokens are encrypted before storage and refreshed by a background job. Disconnecting revokes the token. An expired or revoked connection is shown in the dashboard and makes its tools return `ok: false`, so the agent never claims an action it could not perform.

Booking on a real calendar (synchronous, inside the call).

1. `get_available_slots` combines the tenant's working hours, external free and busy times (cached for up to 60 seconds) and internal appointments.
2. `book_meeting` first reserves the slot in the database (a unique index prevents double booking), marked pending, with an idempotency key made from the call and the slot.
3. It then creates the external event with a short timeout (proposed 3 seconds).
4. On success the reservation is confirmed and stores the external event id. On failure the reservation is released and the tool returns `ok: false`, so the agent offers another time or a callback.

CRM and messaging (asynchronous, through a transactional outbox).

- When a tool or the post-call job changes data, it writes an `outbox_messages` row in the same database transaction (kind, tenant, payload, idempotency key, status, attempts, next attempt time).
- A worker claims due rows with `FOR UPDATE SKIP LOCKED`, calls the adapter, then marks the row done or schedules a retry with exponential backoff and jitter. After a set number of attempts (proposed 8) the row is dead-lettered and shown in the dashboard as a sync issue.
- Delivery is at least once. A mapping table from Aurevia ids to CRM ids plus idempotency keys makes the effect once.
- Per-tenant, per-vendor rate limits keep Aurevia inside each vendor's limits.

Follow-up messages. The `send_followup` tool picks an approved template and fills its fields. The server checks that the template belongs to the tenant and channel, validates the field types and lengths, and asks the compliance gate to approve the channel (consent for that purpose, do-not-disturb for promotional messages, sending hours). Only then is an outbox row written. Status webhooks update delivery records. An opt-out reply such as STOP is handled by the inbound webhook and writes a do-not-contact record at once.

After the call (a job, each step idempotent by call id). Finalise the transcript, write a summary (fast model), extract memory facts (exists), score the lead, create CRM outbox rows, and create the follow-up plan from the outcome (booked: confirmation and reminders; interested: a message and a retry; no answer: a retry within policy). A failure in one step does not block the others.

Public API and webhooks.

- Versioned under `/api/v1`, documented from the generated OpenAPI file.

- API keys are stored hashed, shown once, carry scopes and a per-key rate limit, and can be revoked.
- Outgoing webhooks (`call.completed`, `meeting.booked`, `lead.updated`, `message.delivered`) travel through the same outbox. Each is signed with HMAC-SHA256 over a timestamp and the body, with a five-minute replay window and key rotation.

Compliance gate and call safety

Structure (exists). The only way to place an outbound call is `telephony/service.py dial()`, which requires an `Approval`, and only `ComplianceGate.check_outbound` can create one. Architecture tests enforce both. A database constraint requires every outbound call row to reference the decision that allowed it, and one decision allows exactly one call. Every check runs every time and the decision record stores each result, the facts seen and the policy version, whether the call was allowed or not.

Policy (exists). Rules live in immutable, versioned policy files (the draft is `india-2026-10-draft-1`, identified by a SHA-256 hash). Live calling is impossible until the platform operator records that counsel reviewed that exact version. A tenant may make rules stricter, never looser.

Check	Blocks when	Status
Mode	test mode and not a registered test number; live mode and unreviewed policy	exists
Number	missing, invalid, or not a +91 number	exists
Lead	archived, or marked not interested	exists
Do-not-call	on the tenant's list, including in-call requests	exists
Consent	no valid express consent, or enquiry older than 30 days, for this purpose	exists
Do-not-disturb	registered, or status unknown	exists; the real registry lookup is new
Calling window	outside 09:00 to 21:00 India time	exists

Check	Blocks when	Status
Attempts	more than 2 a day or 6 a week per number	exists
Caller ID	no active, registered 140-series (promotional) or 160-series (service) number	exists
Campaign	not active, or auto-dial not enabled in both the server and the campaign	exists for live calls
Plan and budget	plan limit reached, or the tenant's cost cap is spent	new
Pacing	tenant or global concurrency or dial-rate limit reached	new
Opening requirements	the policy requires an AI notice or recording notice and the agent setup cannot supply it	new

The values are a draft from the product brief; counsel confirms or changes them, and every change becomes a new version.

Do-not-disturb registry (new). An adapter behind the existing `DndRegistry` interface returns registered, not registered or unknown, with a cache whose lifetime counsel sets. A lookup error returns unknown. Unknown blocks. Express consent may override a registration only where the policy says so.

Kill switches (new). Four levels, all stored in the database and read by the gate and by the dialer on every cycle: platform-wide, per tenant (suspend exists), per campaign (pause exists) and per number. A switch stops new calls at once and lets calls in progress finish; an emergency action also ends live rooms.

Safety during a call.

- *Opt-out.* `request_do_not_call` writes the record immediately, the agent closes politely, and the gate blocks every later call.
- *Human requested or sensitive case.* `flag_for_handoff` notifies the right colleague with a summary. A wrong number, a child or a distressed person ends the call politely and records the reason.

- *Voicemail and answering machines.* Detect from the carrier's signal or a short classifier on the first seconds of audio. Default: end without leaving a message, mark it, and retry inside the attempt limits. Leaving a recorded message needs counsel's approval.
- *Disclosure.* The agent says it is an AI whenever asked, and in the opening line if the policy requires it.
- *Pacing.* Concurrency and dial rate caps protect carrier standing and caller-ID reputation, and keep the abandoned-call rate low.

Security and privacy

The project's own threat model is `docs/04-compliance-security/security-threat-model.md`. This section states the design rules; items marked new are additions.

Identity and access

- Passwords are hashed with Argon2. Access tokens are short-lived; refresh tokens rotate, and reuse of an old one revokes the whole session (exists). Sign-in, sign-up, refresh and invitation endpoints are rate limited per address and per email (exists).
- Permissions are explicit: calls, leads, privacy, campaigns, agents, numbers, compliance, team, audit, analytics (exists). New: learning import, review and publish (three separate permissions), integrations, billing, API keys. Nobody can grant more than they hold.
- New: two-factor sign-in for owners and admins, and API keys that are hashed, scoped and revocable.
- The voice worker authenticates with a per-call token passed through the media server's dispatch. It works only for that call's internal endpoints; a user token or another call's token is rejected (exists).

Secrets and encryption

- All keys are secret-typed settings, never returned by an API, redacted from logs and tracebacks, and kept only in a root-only environment file or the host's secret manager (exists). Vendor speech keys exist only in the worker.
- Everything outside the server travels over TLS. Postgres and Redis are on a private network only.
- New: disk encryption on the server and backups; integration tokens encrypted at the application level with a key held outside the database.

Untrusted input (new rules for new inputs)

Prospect speech, imported transcripts, uploaded knowledge documents, CRM fields and calendar event titles are all data, never instructions.

- They enter the prompt only in clearly delimited data sections, length limited, and framed as reference, never as instructions (memory notes already work this way).
- The tenant and the lead for a tool call come from server state, never from the model.
- Tools are limited by stage. For example, `book_meeting` is offered only in the next-step stage, so a spoken instruction cannot invoke it at the start of a call.
- An injection test set runs against every new input path in the nightly evaluation.

Abuse prevention

- A new tenant starts in test mode and cannot dial real numbers until the operator verifies the business (new).
- Plan limits, pacing, daily caps and kill switches (see the compliance section) bound what a misused account can do.
- Acceptable-use terms forbid calling numbers without consent or on the do-not-disturb list, and forbid deceptive scripts.

Audit

Audit events form a hash chain that can be verified through the API (exists). New: a scheduled daily verification that alerts on a break, and audit entries for every integration connection, playbook approval, kill switch and platform-admin action. Every tool execution already writes its own audit row.

Privacy (India's DPDP Act, subject to counsel)

- Collect and keep only what the call needs; transcripts are purged after their retention period (exists).
- Redact personal data before text reaches the learning pipeline, and from prompts where the call does not need it (new).
- Consent records, data export and erasure for a person (exists for leads; extended to new tables).
- Publish a sub-processor list. Ask counsel about sending personal data to speech and model vendors outside India.
- A named grievance contact and a written breach-response plan (new).

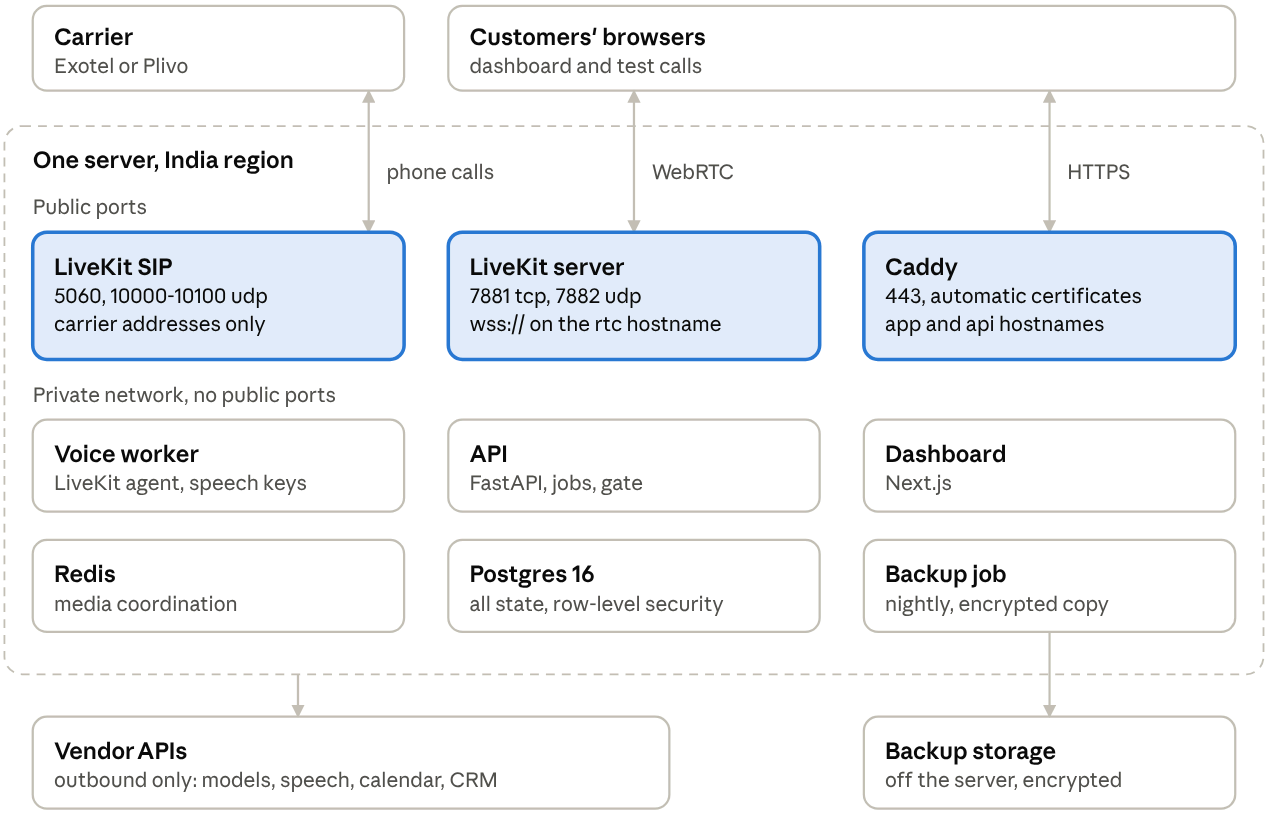
Supply chain and hosting

Locked dependencies (`uv.lock`, `package-lock.json`), automated vulnerability and secret scanning in CI, small container images that run as a non-root user, SSH limited to Virat's address, and only ports 80, 443 and the media ports open to the internet (new SIP ports only to the carrier's addresses).

Telephony, media and deployment

The pilot runs on one server in an Indian region with Docker Compose and Caddy, as decided in `docs/05-operations/hosting-plan.md` (Option A). Provider, domain and budget are still open. The current `docker-compose.yml` is a development file and must not be exposed as it is.

Only three services face the internet; everything else stays private



Blue boxes are the only ones reachable from the internet. SSH (port 22) is open to Virat's address only.

single-server topology · 3 public services

Port	Service	Open to
22	SSH	Virat's address only

Port	Service	Open to
80, 443	Caddy: HTTPS front door for the dashboard, API and media address	Everyone
7881 tcp, 7882 udp	LiveKit media for browser and phone calls	Everyone
5060 and 10000 to 10100 udp	LiveKit SIP and its media range, only once telephony starts	The carrier's addresses only
Postgres, Redis, API, dashboard, worker	Internal container network	Nobody outside the server

Phone call path (exists, untested on the real network).

- *Outbound.* The compliance gate approves and records a decision. The API creates the call row, a media room and the worker dispatch. LiveKit SIP dials through the carrier. When the call is answered, the worker starts the conversation and speaks the greeting. A busy, unanswered or failed dial marks the call failed and closes the room.
- *Inbound.* The carrier forwards to LiveKit SIP, which creates a room. A signed webhook tells the API, which finds the tenant by the dialed number, creates the call and dispatches the worker.
- *Switching carriers is configuration.* The carrier is only a SIP trunk set up in LiveKit, behind the telephony adapter.
- *Phone audio* is narrow-band (8 kHz) and noisy. The speech model setting is chosen for telephone audio and measured on real calls (WP-14.4). Echo and barge-in are checked on a real network.
- *Limits to raise before the pilot.* The carrier's concurrent-call limit, the number pool and registered caller IDs, and each speech and model vendor's concurrency limit.

Environments. Local (development compose), staging and production share one shape and differ only in configuration. Staging is built first and the pilot runs from it. Provider choice, keys, hostnames and cost caps are environment settings; nothing environment-specific is in code.

Build and release.

- GitHub Actions already runs backend, worker and dashboard checks on every pull request. Add image builds tagged by commit.

- Release: pull the tagged images on the server, run database migrations as the schema owner in an explicit step, start the services, check the health endpoint, and keep the previous tag for an immediate rollback.
- Releases happen only when Virat says so; no pipeline deploys on its own (a project rule).

Backups and recovery (scripts exist). A nightly encrypted dump copied off the server, with a restore check that proves the copy loads. Proposed retention: 14 daily and 8 weekly copies. The recovery objectives and the quarterly restore drill are in `docs/05-operations/disaster-recovery.md`.

What changes when the load grows. See the capacity section: first a larger server, then workers on their own hosts, then managed Postgres with a standby, then a second media region. LiveKit Cloud's managed media is the alternative if running the media ports on one server becomes painful.

Cost metering and capacity

Metering (builds on `usage/`). Every provider call writes a usage event with the tenant, call, provider, kind and quantity. Kinds: speech-to-text seconds, text-to-speech characters, model input tokens (cached and uncached) and output tokens, phone seconds (both talk seconds and billed pulse seconds), cached-audio characters saved, and messages by channel.

Prices. A table of immutable price versions holds the unit price per provider and kind with an effective date; prices are never hard-coded. Items priced in dollars carry the exchange rate version used. A cost event is quantity times the price in force when it happened, stored in integer micro-rupees.

```
C_{call} = \sum_{i} q_i \, p_i + o \cdot t_{talk}
```

Here q is the quantity of usage kind i, p its unit price, o a configurable hosting overhead per talk minute, and t is the call's talk time. The headline figures follow from it:

- cost per talk minute = C divided by talk minutes
- cost per booked meeting = the summed cost of a campaign's calls divided by its meetings booked (and, separately, attended)
- gross margin per tenant = revenue minus cost

Phone cost uses billed pulses, because a carrier that bills in 60-second blocks charges a full minute for a 20-second call.

Budget guards (new). A monthly cost cap per tenant. The gate blocks new calls when it is spent, and the dashboard warns at 80%. A platform-wide spend alert protects Virat's own provider bills.

Capacity. One call holds one worker session (voice activity detection plus two streaming connections) and short, input-output-bound requests to the API. Real per-call CPU and memory are not known yet, so the first task is to measure them.

- *Capacity test:* with the simulator, ramp 5, 10, 20 and 40 simultaneous calls on the pilot server, recording CPU, memory, network, median and p95 reply time and error rate. Capacity is the highest level where p95 stays within target and CPU stays under 70% (*proposed*).
- *Worked example:* 20 simultaneous lines for 8 effective hours a day at 60% use is about 5,760 call minutes a day, or roughly 144,000 a month. At 4,500 talk minutes per human caller per month (the assumption in the cost analysis) that equals about 32 human callers.
- *Limits beyond the server:* each vendor limits concurrent streams or requests by plan (speech, voice, model), and the carrier limits concurrent calls. Raise each limit before it becomes the bottleneck.

Scaling path (only when measurements demand it).

1. A larger single server.
2. Voice workers on their own hosts; they are stateless, and the media server dispatches calls to whichever worker is free.
3. Managed Postgres with a standby.
4. A second region for media. Each step is a configuration and infrastructure change, not a rewrite, because every part already talks over network interfaces and adapters.

Observability, targets and failure modes

What to observe

- One trace id per call (the call id) across the worker, the API and every vendor request, so a bad call can be replayed end to end.
- Structured logs with no personal data or secrets: phone numbers masked to the last four digits, no transcript text, no vendor error text (exists in part).

- Metrics: time at each stage of each turn (exists in `voice/metrics.py`), vendor error rates, call outcomes, active calls, outbox queue depth and dead letters, cost per minute, and cost per hour.
- Health: `/api/v1/health/ready` exists; add per-dependency checks (database, media server, each vendor).

Alerts that page Virat (thresholds *proposed*)

- Call setup success under 95% over 15 minutes.
- Median reply time over 1.5 s, or p95 over 3 s, for 10 minutes.
- A vendor's error rate over 10% for 5 minutes.
- Any dead-lettered outbox row.
- Cost per hour over twice its baseline.
- A failed audit-chain verification, a missed backup, or disk use over 80%.

Failure modes and the designed response

Failure	What the system does	What the prospect or customer sees
Model slow	After the first-token deadline, plays one bridge sound, then falls back to the next model	A short pause, then a normal reply
Model down	Circuit breaker skips it for 60 s and uses the fallback; if all fail, apology line, callback scheduled, call ends	A polite end and a promised callback
Text-to-speech fails	One retry on the fallback voice; browser calls show the text and a retry button; phone calls end after repeated failures with the apology line	Voice may change briefly; never silence on a phone call
Speech-to-text fails	Fallback vendor chosen at call start; mid-call, ask once to repeat, then apology and callback	"Could you say that again?"

Failure	What the system does	What the prospect or customer sees
Carrier outage or high dial failure	Campaign pauses automatically when failures pass a threshold (*proposed*: 30% of the last 10 dials); alert	Customer sees a paused campaign and the reason
Media server down	Calls fail at setup, restart policy brings it back, campaigns pause, alert	No calls until recovery
Database down	API returns 503, calls cannot continue, restore runbook starts	Apology and callback where possible
Calendar or CRM down	The booking tool returns `ok: false` and the agent offers a callback; CRM and message rows wait in the outbox and retry	Booking confirmed later by message
Do-not-disturb lookup fails	Status is unknown, so the call is blocked	Fewer calls placed, never an unlawful one
Worker crashes mid-call	The room closes, the webhook marks the call failed, the lead gets a callback, nothing is repeated because tools are idempotent	Call drops; a callback follows
Duplicate tool or webhook delivery	Answered from the stored audit row or idempotency key	Nothing happens twice
Cost spike or runaway campaign	Budget guard blocks new calls, alert, kill switch available	Customer sees the cap reached

Backups and recovery. Scheduled database backups and a regular restore check (scripts exist: `scripts/backup.sh`, `scripts/restore_check.sh`). Recovery time and data-loss limits are written in `docs/05-operations/disaster-recovery.md` and tested by a restore drill each quarter.

Open design decisions

Each row needs a decision before the work it blocks. "Recommended default" is where I would start if no evidence says otherwise; the last column says what settles it.

Decision	Options	Recommended default	Settled by
How to play listening sounds during the prospect's speech	Guarded `session.say`; a second mixed audio source	Whichever the spike proves keeps the prospect's turn intact	WP-11.1 spike
Turn detection	Fixed, dynamic endpointing, turn-detection model	Dynamic, then compare with the model	WP-10.4 benchmark
Fast and strong models	Flash-Lite class, Flash class, Claude Haiku class	Cheapest model that passes all eight scenarios	WP-10.2 bake-off
Second speech-to-text and text-to-speech vendors	Several, including Indian-language specialists	One per layer, chosen by quality per rupee	WP-13.2 bake-off
Knowledge retrieval store	Postgres full-text search, Postgres with a vector extension, a separate vector database	Postgres, so row-level security and one database are kept	Build WP-16.2
Background jobs (outbox, follow-ups, post-call)	Loops inside the API process; a separate queue system; Postgres rows claimed with `SKIP LOCKED`	Postgres outbox with worker loops, so no new infrastructure for the pilot	Revisit if volume demands
Media server	Self-hosted LiveKit; LiveKit Cloud	Self-hosted on the single server; price the managed option	WP-14.2
Observability	Self-hosted metrics, logs and traces; a hosted service	A hosted service on a small plan, fed by OpenTelemetry	Before the pilot

Decision	Options	Recommended default	Settled by
Transient audio storage	Indian-region object storage providers	Only built if recordings are ingested	Phase 12
Phone carrier	Exotel, Plivo, another	The cheapest that passes the phone-audio test	WP-13.6, WP-14.3
Opening AI notice and recording notice	At the start of every call; only when asked	Notice at the start, until counsel says otherwise	Counsel, WP-14.7
Voice cloning	Never; per-speaker written consent	Never	Virat and each speaker
Billing provider	Razorpay; Stripe	Razorpay for India	Phase 16
Per-turn mood source	Words only; words plus tone of voice	Words only first	WP-11.4 results